

Tracing Mutation and Side Effects

Lecture 21

CS 1001

Kirk Duran

Today

- ▶ Reinforce the distinction between **aliasing**, **copying**, and **rebinding**.
- ▶ Predict the effects of functions that receive and mutate lists.
- ▶ Identify when surprising output is caused by shared references and side effects.
- ▶ Repair unintended mutation by copying or constructing independent data.
- ▶ Distinguish accidental mutation from intentional, documented mutation.
- ▶ Simulate changes to immutable strings and tuples by constructing new objects.

Key idea

The central tracing question is not merely “what value does this variable have?” It is also “which object does this name currently refer to?”

Part 1: A Reference-Aware Tracing Model

Why Some Programs Become Hard to Predict

- ▶ With immutable values, a reassignment typically affects only one binding.
- ▶ With mutable lists, several names may refer to the same object.
- ▶ A mutation through one alias changes the object observed through every alias.
- ▶ Function parameters can temporarily become additional aliases to caller-owned lists.

Source of confusion

A line of code may mention only one variable name while changing state visible through several different names.

Three Operations That Must Remain Distinct

Operation	Example	Effect
Rebinding	<code>x = new_value</code>	x refers to another object
Aliasing	<code>y = x</code>	another name refers to the same object
Mutation	<code>x[0] = 7</code>	existing mutable object changes

Key idea

Assignment does not imply copying. Indexed assignment to a list does not rebind the list variable.

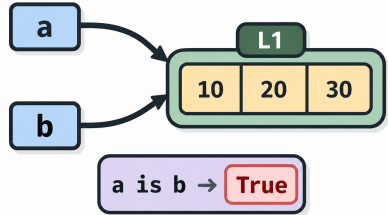
Reminder: Aliasing

Code

```
a = [10, 20, 30]  
b = a
```

After the second statement:

- ▶ there is one list object;
- ▶ both a and b refer to it;
- ▶ a is b is True.



A Reliable Tracing Procedure

For each statement involving a list:

1. Identify the object or value produced by the right-hand side.
2. Determine whether the statement creates a new object, a new alias, or a mutation.
3. Update the name-to-object bindings.
4. Update the state of any mutated object.
5. If a function is called, add its parameter bindings to the same object diagram.

Key idea

Track **bindings** and **object state** separately.

Warm-Up: Predict Before Running

Code

```
first = [2, 4, 6]
second = first
second[1] = 99
print(first)
print(second)
```

In class

Predict both printed lists. Draw the reference diagram before deciding.

Warm-Up: Reveal

Output

```
[2, 99, 6]
```

```
[2, 99, 6]
```

- ▶ `second = first` created an alias, not a copy.
- ▶ `second[1] = 99` mutated the shared list object.
- ▶ The expression `first` observes that same mutated object.

Part 2: Functions Can Create Caller-Visible Side Effects



Prediction 1: A Function Mutates Its Parameter

Code

```
def zero_first(values: list[int]) -> None:
    values[0] = 0

scores = [8, 9, 10]
alias = scores
zero_first(alias)
print(scores)
print(alias)
```

In class

Predict both outputs. At the indexed assignment, which names refer to the same list object?

Prediction 1: What Actually Happens

During the function call:

scores ↘ alias → [8, 9, 10] ← values

After `values[0] = 0`:

[0, 9, 10]

Output

[0, 9, 10]

[0, 9, 10]

Key idea

Parameter binding can introduce a temporary alias to a caller-owned mutable object.

The Unexpected Result Is a Side Effect

Definition

A **side effect** is an observable change to program state that occurs in addition to producing a return value.

In the previous function:

- ▶ the function returns `None`;
- ▶ the caller's list changes anyway;
- ▶ the change remains after the call frame ends.

Important

The problem is not that mutation occurred. The problem is that mutation may be surprising if the function's purpose does not clearly promise it.

Repair Strategy 1: Copy Before Calling

Code

```
scores = [8, 9, 10]  
working = scores.copy()  
zero_first(working)
```

scores → [8, 9, 10]

working → [0, 9, 10]

Key idea

Copying breaks the alias before mutation occurs.

Repair Strategy 2: Copy Inside the Function

Code

```
def with_zero_first(values: list[int]) -> list[int]:  
    result = values.copy()  
    result[0] = 0  
    return result
```

- ▶ The parameter `values` still aliases the caller's list.
- ▶ `result` refers to a distinct list object.
- ▶ Mutation is applied only to the new list.

Key idea

A “return a modified copy” design makes the state change explicit in the returned value.

Part 3: Harder Aliasing Traces

Prediction 2: A Local Alias Is Still an Alias

Code

```
def adjust(values: list[int]) -> list[int]:  
    local = values  
    local[1] = local[1] + 10  
    return local  
  
data = [1, 2, 3]  
result = adjust(data)  
result[-1] = 99  
print(data)  
print(result)
```

In class

Predict both outputs. Is `local = values` a copy operation?

Prediction 2: Reveal and Diagnose

Output

```
[1, 12, 99]
```

```
[1, 12, 99]
```

- ▶ `local = values` creates another name for the same object.
- ▶ The function mutates that shared object at index 1.
- ▶ Returning `local` returns a reference to the same list.
- ▶ Therefore `result` becomes yet another alias.
- ▶ Mutating `result[-1]` also changes what `data` observes.

Fix Prediction 2

Replace the aliasing assignment:

Problem

```
local = values
```

with an independent copy:

Repair

```
local = values.copy()
```

Then:

data → [1, 2, 3]

result → [1, 12, 99]

Prediction 3: Concatenation Can Break an Alias

Code

```
def add_marker(values: list[int]) -> list[int]:  
    temp = values  
    temp = temp + [0]  
    temp[0] = 9  
    return temp  
  
original = [1, 2, 3]  
changed = add_marker(original)  
print(original)  
print(changed)
```

In class

Does original change? Identify the exact statement where the reference relationship changes.

Prediction 3: Reveal

Output

```
[1, 2, 3]  
[9, 2, 3, 0]
```

- ▶ Initially, `temp` aliases `values`.
- ▶ `temp + [0]` constructs a new list.
- ▶ The assignment then rebinds `temp` to that new list.
- ▶ The later indexed mutation affects only the new list.

Key idea

An alias can cease to be an alias when one name is rebound to a different object.

Compare: + Versus extend

Operation	Creates new list?	Mutates original?
<code>new = values + [0]</code>	yes	no
<code>values.extend([0])</code>	no	yes

Tracing consequence

With aliases, `extend` is visible through every reference to that list. Concatenation produces a separate list object.

Prediction 4: Copy, Then Mutate

Code

```
def adjust_copy(values: list[int]) -> list[int]:  
    result = values[:]  
    result[0] = 50  
    return result  
  
a = [5, 6, 7]  
b = a  
c = adjust_copy(b)  
b[1] = 60
```

In class

Predict the final values observed through a, b, and c.

Prediction 4: Reveal

Name	Final value observed	Object
a	[5, 60, 7]	L1
b	[5, 60, 7]	L1
c	[50, 6, 7]	L2

Key idea

The copy separates c from the aliased pair a/b; it does not retroactively remove aliasing between a and b.

Part 4: Techniques for Avoiding Unintended Shared Mutation

Several Ways to Create an Independent List

For the flat lists of primitive values used in this unit, each expression below creates a distinct list object:

Examples

```
copy1 = original.copy()  
copy2 = original[:]  
copy3 = list(original)
```

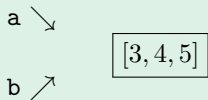
For a known fixed structure, explicit recreation is also possible:

Explicit recreation

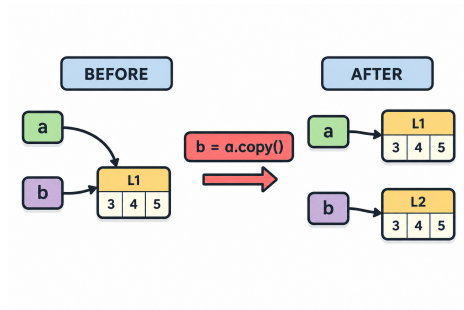
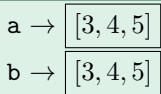
```
copy4 = [original[0], original[1], original[2]]
```

Mental Model: Copying Breaks the Shared Reference

Before copy



After `b = a.copy()`



Technique 1: `list.copy()`

Code

```
original = [10, 20, 30]
working = original.copy()
working[0] = 99
```

Result

```
original == [10, 20, 30]
working == [99, 20, 30]
```

Key idea

`copy()` directly communicates the programmer's intent to create another list object.

Technique 2: Full Slicing

Code

```
original = [10, 20, 30]
working = original[:]
working[-1] = 0
```

A full slice selects all elements and constructs a new list.

Result

```
original == [10, 20, 30]
working == [10, 20, 0]
```

Technique 3: `list(original)`

Code

```
original = [1, 2, 3]
working = list(original)
working.remove(2)
```

- ▶ `list(original)` constructs a new list from the supplied sequence.
- ▶ The new list has equal contents initially but distinct identity.

Identity check

```
working == original is initially True.
working is original is False.
```

Technique 4: Explicit Reconstruction

When the structure is small and fixed, the new value can be written explicitly.

Code

```
point = [4, 7, 9]
changed = [point[0], 100, point[2]]
```

- ▶ `point` is unchanged.
- ▶ `changed` is a new list with the desired contents.
- ▶ The intended change is represented as construction, not mutation.

Where Should the Copy Happen?

There are two legitimate designs:

Caller protects itself

```
working = data.copy()  
mutate(working)
```

Useful when the called function is intentionally mutating.

Function protects caller

```
def transformed(data):  
    result = data.copy()  
    ...  
    return result
```

Useful when the function promises not to modify its argument.

Part 5: Mutation Is Not Inherently Bad

Intentional Mutation Is a Valid Design Choice

Key idea

Mutation is appropriate when changing an existing object is the intended behavior of the operation.

Examples already used in this unit:

- ▶ `append` intentionally grows a list;
- ▶ `remove` intentionally deletes an element;
- ▶ `reverse` intentionally changes list order;
- ▶ indexed assignment intentionally replaces an element.

The important question is whether that state change is part of the function's clear contract.

Intentional Example: Swap the Endpoints

Code

```
def swap_ends(values: list[int]) -> None:
    first = values[0]
    values[0] = values[-1]
    values[-1] = first
```

Use

```
numbers = [10, 20, 30, 40]
swap_ends(numbers)
```

After the call, `numbers` is intentionally `[40, 20, 30, 10]`.

Why a None Return Can Be Appropriate

For a function whose primary purpose is mutation:

- ▶ the important observable result is the changed object;
- ▶ returning `None` emphasizes that the function acts through a side effect;
- ▶ the caller should not expect a separate transformed list.

Contrast

`swap_ends(values) -> None`

`swapped_ends(values) -> list[int]`

mutates supplied list

returns a new list

Two Clear Interface Designs

Design	Argument	Returned value
Mutating operation	may change	often None
Transform-and-return	unchanged	new value/object

Avoid ambiguity

A function is easier to reason about when its name, purpose statement, tests, and implementation agree about whether mutation is expected.

Part 6: Simulated Mutation for Immutable Sequences



Strings Cannot Be Mutated In Place

Invalid code

```
text = "cat"  
text[0] = "b"
```

Python raises a `TypeError` because strings are immutable.

Key idea

To represent a changed string, construct a new string and bind a name to it.

Simulated String Mutation

Construct a new string

```
text = "cat"  
changed = "b" + text[1:]
```

Now:

text → "cat"

changed → "bat"

Reusing the same variable name is also possible:

Rebinding

```
text = "b" + text[1:]
```

This is construction plus rebinding, not mutation of the original string.

Tuples Are Immutable Sequences

Example

```
point = (10, 20, 30)
```

Tuples support familiar sequence operations:

- ▶ indexing: `point[0]`;
- ▶ negative indexing: `point[-1]`;
- ▶ slicing: `point[1:]`;
- ▶ length and membership checks.

But indexed assignment is not permitted:

Invalid

```
point[1] = 99
```

Simulated Tuple Mutation

Suppose we want to replace the middle element:

Construct a new tuple

```
point = (10, 20, 30)
changed = point[:1] + (99,) + point[2:]
```

- ▶ `point` remains `(10, 20, 30)`.
- ▶ `changed` is `(10, 99, 30)`.
- ▶ `(99,)` is a one-element tuple; the comma is required.

Returning a Changed Tuple

Code

```
def replace_middle(values: tuple[int, int, int],  
                  new_value: int) -> tuple[int, int, int]:  
    return values[:1] + (new_value,) + values[2:]
```

- ▶ The input tuple is never mutated.
- ▶ The function returns a different tuple containing the requested change.
- ▶ This is the immutable analogue of “copy, change, return.”

Part 7: Integrated Tracing Challenge

Integrated Prediction

Code

```
def bump_first(values: list[int]) -> None:
    values[0] = values[0] + 1

def adjusted(values: list[int]) -> list[int]:
    result = values[:]
    result[-1] = 0
    bump_first(result)
    return result

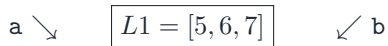
a = [5, 6, 7]
b = a
c = adjusted(b)
b[1] = 9
```

In class

Predict the final values of a, b, and c. Draw object labels before tracing statements.

Integrated Trace: Object Structure

After `b = a`:



Inside adjusted:

- ▶ parameter values aliases L1;
- ▶ `result = values[:]` creates L2;
- ▶ `result[-1] = 0` changes only L2;
- ▶ `bump_first(result)` temporarily aliases L2 through its parameter and mutates L2.

`L1 = [5, 6, 7]` `L2 = [6, 6, 0]`

Integrated Trace: Final Result

After `c = adjusted(b)`:

`a, b` $\rightarrow$ `L1` = [5, 6, 7]

`c` $\rightarrow$ `L2` = [6, 6, 0]

Then `b[1] = 9` mutates `L1`.

Final values

`a` == [5, 9, 7]

`b` == [5, 9, 7]

`c` == [6, 6, 0]

Debugging Unexpected Mutation

When a list changes unexpectedly, ask these questions in order:

1. Which statement first changed the object's state?
2. Which names referred to that object at that moment?
3. Did a function parameter alias a caller-owned list?
4. Was the mutation intentional according to the function's contract?
5. If not, where should an independent object have been created?

Key idea

Locate the first unintended mutation, not merely the final surprising print statement.

Common Incorrect Mental Models

- ▶ **Incorrect:** “Assignment copies the list.”
Correct: ordinary assignment copies the reference/binding relationship.
- ▶ **Incorrect:** “The function changed the variable in the caller.”
Correct: the function mutated an object that both caller and parameter referenced.
- ▶ **Incorrect:** “Returning the list makes a copy.”
Correct: returning an existing list returns a reference to that object.
- ▶ **Incorrect:** “Every operation that looks like a change is mutation.”
Correct: strings and tuples require construction of new objects.

Takeaways

- ▶ Aliasing means multiple names refer to one object.
- ▶ Mutation changes an existing object and is visible through all of its aliases.
- ▶ Function parameters can alias mutable objects owned by the caller.
- ▶ Unexpected shared mutation is a side effect worth diagnosing explicitly.
- ▶ Use `copy()`, full slicing, `list(...)`, or explicit reconstruction when independent list state is required.
- ▶ Intentional mutation is appropriate when it is part of the function's contract.
- ▶ Strings and tuples are immutable; represent changes by constructing and rebinding new values.

Key idea

When tracing mutable data, follow the objects first and the variable names second.